MACHINE LEARNING ASSIGNMENT – 8

1. B
2. A
3. C
4. C
5. A
6. C
7. C
8. A,D
9. B,D
10. A,D

11. In which situation One-hot encoding must be avoided? Which encoding technique can be used in such a case?

One-hot encoding can be avoided in some cases:

High Cardinality Features:
High cardinality features are those with a large number of unique values. One-hot encoding generates a binary feature for each unique category, which can lead to a large number of features when applied to high cardinality features. In such cases, one-hot encoding may lead to a sparse dataset with too many features, which can cause computational and storage problems.

Non-categorical Data: One-hot encoding should only be used for categorical data. If a continuous or numerical feature is one-hot encoded, it can lead to a large number of binary features, which can make the dataset difficult to manage.

Time-series Data: In time-series data, one-hot encoding can result in the loss of temporal information, and it may not be the best choice.

12. In case of data imbalance problem in classification, what techniques can be used to balance the dataset? Explain them briefly.

Data imbalance is a common problem in classification tasks where one class has significantly more samples than the other(s). This can lead to biased models that perform poorly on the minority class. Here are some techniques that can be used to balance the dataset:

Undersampling: This technique involves reducing the number of samples in the majority class to match the number of samples in the minority class. This can be done randomly or through more sophisticated methods, such as Tomek Links or Cluster Centroids.

Oversampling: This technique involves increasing the number of samples in the minority class to match the number of samples in the majority class. This can be done through duplication of samples or through more advanced methods such as Synthetic Minority Over-sampling Technique (SMOTE), Adaptive Synthetic Sampling (ADASYN) or Random Oversampling Minority (ROM).

Combination of Undersampling and Oversampling: This approach involves combining the above techniques to balance the dataset. For example, one can first undersample the majority class and then oversample the minority class.

Class weighting: This technique involves assigning a weight to each class during the training process. This will give more importance to the minority class and penalize the misclassification of the minority class more severely.

Anomaly detection: This technique involves identifying and removing any outliers or noise in the dataset, which may be contributing to the class imbalance.

Using different algorithms: Some algorithms are more sensitive to class imbalance than others. Trying different algorithms that are known to handle class imbalance well can also help.

13. What is the difference between SMOTE and ADASYN sampling techniques?

SMOTE (Synthetic Minority Over-sampling Technique) and ADASYN (Adaptive Synthetic Sampling) are two commonly used oversampling techniques to address the problem of class imbalance in classification problems.

Although both SMOTE and ADASYN are based on synthetic data generation, there are some differences between the two approaches:

Synthetic sample generation: SMOTE generates synthetic samples by interpolating between existing minority class samples, while ADASYN generates synthetic samples by adjusting the distribution of the minority class in each iteration.

Weights calculation: ADASYN assigns weights to each sample in the minority class, based on the distribution of samples in the feature space, while SMOTE does not assign any weights.

Sampling strategy: ADASYN generates more synthetic samples near the decision boundary, while SMOTE generates synthetic samples uniformly across the feature space.

Performance: ADASYN may perform better than SMOTE when the imbalance ratio is very high or when the decision boundary between classes is complex and non-linear.

Computation complexity: ADASYN can be computationally more expensive than SMOTE because it requires calculating weights and distances for each minority class sample.

In summary, SMOTE and ADASYN are two effective oversampling techniques for addressing the class imbalance problem in classification. SMOTE generates synthetic samples by interpolating between existing minority class samples, while ADASYN generates synthetic samples by adjusting the distribution of the minority class in each iteration. Both techniques have their strengths and weaknesses, and the appropriate technique should be chosen based on the specific dataset and problem at hand.

14. What is the purpose of using GridSearchCV? Is it preferable to use in case of large datasets? Why or why not?

GridSearchCV is a function in scikit-learn that performs an exhaustive search over a specified parameter grid for the best combination of hyperparameters for a given estimator.

The purpose of using GridSearchCV is to find the optimal hyperparameters for a machine learning model that maximizes its performance on a given metric (such as accuracy or F1-score).

GridSearchCV is particularly useful when we have a large number of hyperparameters to tune, and we want to avoid the time-consuming process of manually trying out different combinations.

GridSearchCV allows us to specify a range of values for each hyperparameter and automatically generates all possible combinations, then evaluates the estimator using cross-validation on each set of hyperparameters to find the best combination.

In terms of performance, GridSearchCV can be computationally expensive, especially for large datasets, as it performs an exhaustive search over all possible hyperparameter combinations. However, the computational cost can be mitigated by specifying a smaller range of hyperparameter values, reducing the number of hyperparameters to tune, or using parallelization to distribute the computations across multiple cores.

In summary, GridSearchCV is a powerful tool for hyperparameter tuning that automates the process of finding the optimal hyperparameters for a machine learning model. While it may be computationally expensive for large datasets, it can still be useful in such cases by using appropriate optimizations such as reducing the range of hyperparameters or using parallelization.

15. List down some of the evaluation metric used to evaluate a regression model. Explain each of them in brief

Mean Squared Error (MSE): MSE is a measure of the average squared difference between the predicted and actual values. It is calculated as the average of the squared differences between the predicted and actual values. The lower the MSE, the better the model's performance.

Root Mean Squared Error (RMSE): RMSE is the square root of the MSE and is often used to express the error in the same units as the target variable. It is a measure of the average distance between the predicted and actual values. The lower the RMSE, the better the model's performance.

Mean Absolute Error (MAE): MAE is a measure of the average absolute difference between the predicted and actual values. It is calculated as the average of the absolute differences between the predicted and actual values. The lower the MAE, the better the model's performance.

R-squared (R^2): R-squared is a measure of the proportion of the variance in the target variable that is explained by the model. It ranges from 0 to 1, with higher values indicating a better fit. A value of 1 indicates that the model perfectly fits the data, while a value of 0 indicates that the model does not explain any of the variability in the data.

Adjusted R-squared: Adjusted R-squared is a modified version of R-squared that takes into account the number of predictors in the model. It penalizes models that add unnecessary predictors and gives a more accurate measure of the model's fit.